disorder domain. A dangerous state where we really don't understand or can't make sense of the situation we are in. Our goal is to get out of this domain. 2. One of the domains in the Cynefin framework. See also *Cynefin*. Contrast with *chaotic domain, complex domain, complicated domain, simple domain.*

done. See *definition of done*.

dot voting. A technique that allows participants to vote their preferences among a set of items by placing a colored dot on items that they believe are higher priority than other items. Items with more dots are higher priority than items with fewer dots. This technique is frequently used during the sprint retrospective activity. See also *sprint retrospective.*

E

economic filter. The decision criteria used by an organization to evaluate the economics of a proposed product in order to decide whether or not to fund it. Contrast with *strategic filter.*

emergence. 1. Individual, localized behavior that aggregates into global behavior that is disconnected from its origins. 2. An attribute of complex adaptive systems. 3. When applied to software development, recognizing that it is not possible to a priori determine the correct set of features, designs, or plans. Instead, over time as more information is learned, important information will emerge from the experience gained on prior work. See also *complex adaptive system*.

emergent opportunity. An opportunity that was previously unknown, or was deemed sufficiently unlikely to occur and therefore not worth spending money on at the time.

emotions seismograph. A graphical representation of the emotional ups and downs of team members over the course of a sprint. A technique frequently used during the sprint retrospective activity. See also *sprint retrospective*.

empirical process control. A style of work that leverages the principles of inspection, adaptation, and transparency. Contrast with *defined process.*

end uncertainty. Uncertainty surrounding what will be built (the product). See also *uncertainty.* Contrast with *customer uncertainty, means uncertainty.*

envisioning. An activity that captures the essence of a potential product and creates a rough plan for the creation of that product. Envisioning begins with the creation of a vision, followed by the creation of a high-level product backlog and frequently a product roadmap. Synonymous with *product planning.* See also *product roadmap.*

epic. A large user story, perhaps a few to many months in size, that can span an entire release or multiple releases. Epics are useful as placeholders for large requirements. Epics are progressively refined into a set of smaller user stories at the appropriate time. See also *feature, progressive refinement, theme, user story.*

essential Scrum. The values, principles, and practices of the Scrum framework combined with rules and proven approaches to applying Scrum practices. See also *approach, practice, rule, Scrum framework*.

estimation. A rough calculation of the value, number, quantity, or extent of something. In Scrum, we estimate the size of portfolio backlog items, product backlog items, and sprint backlog tasks. See also *forecast*.

event timeline. A visual, chronologically ordered depiction of the meaningful events that occurred over a period of time. A common technique used during sprint retrospectives. See also *sprint retrospective*.

exploitation. Making a decision based on the certainty of the information we currently possess. Contrast with *exploration*.

exploration. The act of acquiring or buying knowledge by performing some activity such as building a prototype, creating a proof of concept, performing a study, or conducting an experiment. Contrast with *exploitation*.

external stakeholders. Stakeholders who are typically external to the organization that is developing a product, for example, customers, partners, and regulators. See also *stakeholders.* Contrast with *internal stakeholders*.

Extreme Programming (XP). An agile development approach that is complementary to Scrum. Extreme Programming specifies important technical practices that development teams use to manage the flow of task-level work during sprint execution. See also *agile*.

F

fail fast. A strategy of trying something, getting fast feedback, and then rapidly inspecting and adapting. In the presence of high levels of uncertainty, it is often less expensive to start working on a product, learn whether we made a good decision, and if not, kill it fast before more money is spent. See also *fast feedback, inspect and adapt, pivot*.

fast feedback. A principle that states that feedback today is much more valuable than the same feedback tomorrow, because today's feedback can be used to correct a problem before it compounds into a much larger problem, and provides the ability to truncate economically undesirable paths sooner (to fail faster). See also *fail fast*.

feature. 1. A slice of business functionality that is meaningful to a customer or user. 2. Used by some to mean a medium-size user story that can and will be divided into a collection of smaller user stories that together will be implemented to deliver the value of a feature. See also *theme, user story*.

feature team. A cross-functional and cross-component team that can pull end-customer features from the product backlog and complete them. See also *cross-functional team*. Contrast with *component team*.

fixed-date release. A release that must be delivered on a known future date. The scope of the release, and possibly the cost, needs to be flexible. Contrast with *fixed-scope release*.

fixed-scope release. A release that must have a specific set of features. The date on which the features are delivered and/or the costs are flexible. Contrast with *fixed-date release*.

flow. 1. The smooth, steady movement of work through the development process to ensure that good economic value is delivered. 2. Avoiding idle work in economically sensible ways. 3. The opposite of big batch, big release, and big bang.

forecast. 1. Making statements, predictions, or estimations about events whose actual outcomes have not yet been observed. 2. The 2011 "Scrum Guide" term for what a development team generates during sprint planning. See also *estimation*. Contrast with *commitment*.

framework. See *Scrum framework*.

G

grooming. See *product backlog grooming*.

group. A collection of people who share a common label (the group name) but have not yet formed a team whose members have learned how to work together and trust each other. Contrast with *team*.

H

happened-upon technical debt. A status category for technical debt that represents debt that the development team was unaware existed until it was exposed during the normal course of performing work on the product. Contrast with *known technical debt, targeted technical debt*. See also *technical debt*.

I

ideal day. A unit for estimating the size of product backlog items based on how long an item would take to complete if it were the only work being performed, there were no interruptions, and all resources necessary to complete the work were immediately available. See also *ideal hour*. Contrast with *story point*.

ideal hour. A unit for estimating the size of the design, build, integrate, and test work, represented as sprint backlog tasks. Often referred to as an effort-hour, person-hour, or man-hour. See also *ideal day*.

idle work. Work that is not actively being pursued as it sits in some queue. Contrast with *idle workers*.

idle workers. People who have available capacity to do more work because they are not currently 100% utilized. Contrast with *idle work*.

impediment. A hindrance or obstruction to doing something. Frequently used to describe some issue or blocker that is preventing a team or organization from performing Scrum in an effective way.

implementable story. A user story that is sized small enough to fit nicely within a sprint. Synonymous with *sprintable story*.

incremental development. 1. Development based on the principle of building *some* before building *all*. 2. A staging strategy in which parts of the product are developed and delivered to users at different times, with the intention to adapt to external feedback. See also *iterative and incremental process, iterative development*.

incremental funding. Funding some of the product development without committing to funding all of it. Using incremental funding, we fund just the first small part of the development effort and revisit the funding decision after we have the critical validated learning we are paying to get from the first part. See also *confidence threshold, validated learning*.

information radiator. A visual display that presents up-to-date, sufficiently detailed, and important information to passersby in an easy, self-interpretable format.

innovation accounting. A measurement/accounting system that uses actionable metrics to evaluate how fast we are learning as a critical measure of progress toward converging on a business-valuable result (Ries 2011).

innovation waste. The lost opportunity to create an innovative solution. Frequently occurs when a prescribed solution is provided with a product backlog item.

in-process product. A product that is currently under development, already live in production, or currently being sold. See also *portfolio planning*.

insight backlog. A prioritized list of previously generated insights or process improvement ideas that have not yet been acted upon. The insight backlog is generated and used during sprint retrospectives. See also *sprint retrospective*.

inspect and adapt. 1. A common phase in Scrum that refers to the inspection and adaptation principles of empirical process control. 2. The principle of inspecting a product or process and making adaptations based on what is learned. 3. A key part of the learning loop. See also *adaptation, empirical process control, inspection, learning loop*.

inspection. One of the three pillars of empirical process control, involving thoughtful examination and processing of feedback to make adaptation decisions regarding the process or product. See also *adaptation, empirical process control, transparency.*

integration. The combining of the various components or assets of some or all of a product to form a coherent, larger-scope work product that can be validated to function correctly as a whole. See also *continuous integration*.

internal stakeholders. Stakeholders who are internal to the organization that is developing the product, for example, senior executives, managers, and internal users. See also *stakeholders.* Contrast with *external stakeholders.*

inventory. See *work in process.*

INVEST. An acronym coined by Bill Wake for remembering a set of criteria used to evaluate the quality of user stories. The criteria are *Independent*, *Negotiable*, *Valuable*, *Estimatable*, *Sized correctly* (small), and *Testable*. See also *user story.*

iteration. A self-contained development cycle focused on performing all of the work necessary to produce a valuable outcome. See also *all-at-once development, sprint.*

iterative and incremental process. A style of development that leverages both iterative development and incremental development. See also *incremental development, iterative development.*

iterative development. A planned rework strategy where multiple passes over the work are used to converge on a good solution. See also *incremental development, iteration, iterative and incremental process.*

J

just in time (JIT). A characteristic of a process whereby the assets or activities of a work stream become available or occur just as they are needed.

K

Kanban. An agile approach overlaid on an existing process that advocates visualizing how work flows through a system, limiting the work in process, and measuring and optimizing the flow of work. See also *agile, work in process.*

known technical debt. A status category for technical debt that represents the debt that is known to the development team and has been made visible for future consideration. Contrast with *happened-upon technical debt, targeted technical debt.* See also *technical debt.*

L

last responsible moment (LRM). A strategy of not making a premature decision but instead delaying commitment and keeping important and irreversible decisions open until the cost of not making a decision becomes greater than the cost of making a decision.

learning loop. A feedback loop focused on increasing learning. Generally follows these steps: make an assumption (or set a goal), build something (perform some activities), get feedback on what was built, and then use that feedback to inspect what was done relative to what was assumed.

lifecycle profits. 1. The total profit potential for a product over its lifetime. 2. In the case of portfolio planning, the total profit potential of the entire portfolio rather than a single product.

LRM. See *last responsible moment.*

M

means uncertainty. Uncertainty surrounding how something will be built. See also *uncertainty.* Contrast with *customer uncertainty, end uncertainty.*

minimum marketable features (MMFs). The smallest or minimum set of functionality related to a feature that must be delivered for the customer to perceive value (for it to be marketable). Contrast with *minimum releasable features.*

minimum releasable features (MRFs). 1. The minimum set of features that must be present in a release to make it viable—useful enough to end customers such that they want it and would be willing to pay for it. 2. Features composed from a collection of minimum marketable features. Synonymous with *must-have features.* See also *minimum marketable features.*

minimum viable product (MVP). A product that has just those features that allow the product to be deployed, and no more.

MMFs. See *minimum marketable features.*

MRFs. See *minimum releasable features.*

MVP. See *minimum viable product.*

Musketeer attitude. 1. All for one and one for all. 2. The attitude among members of a team that they are all in the same boat and that they will win or lose together as a team.

must-have features. The set of features that must be present in the upcoming release for the release to be viable. Synonymous with *minimum releasable features.* Contrast with *nice-to-have features, won't-have features.*

N

naive technical debt. A form of technical debt that accrues due to irresponsible behavior or immature practices on the part of the people involved. Contrast with *strategic technical debt, unavoidable technical debt*. See also *technical debt*.

nice-to-have features. Features that are targeted for the upcoming release but could be excluded if there are insufficient resources to finalize their development. Contrast with *must-have features, won't-have features*.

nonfunctional requirement. 1. A requirement that does not relate to functionality but to attributes such as reliability, efficiency, usability, maintainability, and portability, which product backlog items must possess in order to be fully accepted by the stakeholders. 2. Each nonfunctional requirement is a candidate for inclusion in the definition of done. See also *definition of done*.

P

PBI. See *product backlog item*.

persona. 1. A user archetype, synthesized from the ethnographic data of real users, that helps guide decisions about product features, navigation, interactions, and visual design. 2. A fictitious person that is the prototypical instance of a particular user role. See also *user story*.

pigs. A metaphor used by some Scrum teams to indicate that people are invested in the goal of the Scrum team at a commitment level (accountable for the outcome). Most people consider the members of the Scrum team to be pigs. See also *Scrum team*. Contrast with *chickens*.

pivot. 1. To change directions but stay grounded in what we have learned. 2. A structured course correction designed to test a new fundamental hypothesis about a product, strategy, and engine of growth (Ries 2011).

plan-driven process. A style of development that attempts to plan for and anticipate up front all of the features a user might want in the end product and to determine how best to build those features. The work plan is based on execution of a sequential set of work-specific phases. Synonymous with *anticipatory process, predictive process, prescriptive process, sequential process, traditional development process, waterfall process*.

Planning Poker. A consensus-based technique for the relative sizing of product backlog items.

point inflation. The unfortunate behavior of inflating the value of product backlog size estimates in an attempt to conform to or optimize an unwisely conceived measure (such as achieving a target velocity).

portfolio backlog. A backlog composed of products, programs, projects, or high-level epics. See also *portfolio planning.*

portfolio planning. An activity for determining which products (or projects) to work on, in which order, and for how long. Sometimes referred to as portfolio management.

potentially shippable product increment. Results that are completed to a high degree of confidence and represent work of good quality that is potentially shippable to end customers at the end of a sprint. Being potentially shippable does not mean the results will actually be delivered to customers. Shipping is a business decision; potentially shippable is a state of confidence.

practice. The way in which a principle is supported or realized. For example, the principle of demonstrating progress is supported by the sprint review Scrum practice. See *activity, artifact, role, rule*. See also *principle, values.*

precision. How exact an estimate is. For example, saying a product will ship on October 7, 2015, is more precise than saying a product will ship in October 2015. Contrast with *accuracy.*

predictive process. See *plan-driven process.*

prescriptive process. See *plan-driven process.*

principle. A fundamental truth or belief that serves as the foundation for how we approach product development. An example Scrum principle is to demonstrate progress frequently. See also *practice, values.*

principle of least astonishment. Acting or developing work products in a way that is least likely to startle those around you.

product. 1. The result of a product development effort. 2. A good or service consisting of a bundle of tangible and intangible attributes that satisfies consumers and is received in exchange for money or some other unit of value. 3. Typically a longer-lived, more stable artifact against which organizations might conduct one or more projects. See also *product development effort*. Contrast with *project.*

product backlog. A prioritized inventory of yet-to-be-worked-on product backlog items. See also *product backlog item*.

product backlog grooming. The activities of writing and refining, estimating, and prioritizing product backlog items.

product backlog item (PBI). 1. An item such as a feature, defect, or (occasionally) technical work that is valuable from the product owner's perspective. 2. An item in the product backlog. See also *product backlog.*

product development effort. The full scope of work performed to create or enhance a product or service. Contrast with *project.*

product owner. The empowered central point of product leadership. One of the three roles on a Scrum team; the single voice of the stakeholder community to the Scrum team. The product owner defines what to do and in what order to do it. See also *Scrum team*.

product owner proxy. A person enlisted by the product owner to act on his behalf in particular situations. See also *product owner*.

product planning. See *envisioning*.

product roadmap. A description of the incremental nature of how a product will be built and delivered over time, along with the important factors that drive each individual release. Useful when developing a product that will have more than one release. See also *envisioning*.

product vision. A brief statement of the desired future state that would be achieved by developing and deploying a product. A good vision should be simple to state and provide a coherent direction to the people who are asked to realize it. See also *envisioning*.

progressive refinement. To disaggregate, in a just-in-time fashion, large, lightly detailed product backlog items into a set of smaller, more detailed items.

project. 1. A temporary endeavor undertaken to create a unique product, service, or result (PMI 2008). 2. An effort that completes when its objectives have been obtained. Compared with a life of a product, a project is shorter in duration. Frequently multiple projects are performed over the full cradle-to-grave lifecycle of a product. Contrast with *product*.

project chartering. The set of up-front work needed to define a project at a sufficient level of detail that a funding decision can be made. Synonymous with *project inception, project initiation*.

project inception. See *project chartering*.

project initiation. See *project chartering*.

Q

queue. A holding place for items (an inventory) as they wait for the next action in a work stream. See also *inventory, work in process*.

R

refactoring. A technique for restructuring an existing body of code by improving/simplifying its internal structure (design) without changing its external behavior. Refactoring is one of the principal techniques for managing technical debt. See also *technical debt, technical practices*.

relative size measure. A means of expressing the overall size of an item where the absolute value is not considered, but the relative size of an item compared to other items is considered. For example, an item of size 2 is half the size of an item of size 4, but we have no idea how big an item of size 2 or 4 is in some absolute sense. See also *ideal day, story point*.

release. 1. A combination of features that when packaged together make for a coherent deliverable to customers or users. 2. A version of a product that is promoted for use or deployment. Releases represent the rhythm of business-value delivery and should align with defined business cycles.

release goal. A clear statement of the purpose and desired outcome of a release. A release goal is created by considering many factors, including the target customers, high-level architectural issues, and significant marketplace events. See also *release*.

release plan. 1. The output of release planning. On a fixed-date release, the release plan will specify the range of features available on the fixed future date. On a fixed-scope release, the release plan will specify the range of sprints and costs required to deliver the fixed scope. 2. A plan that communicates, to the level of accuracy that is reasonably possible, when the release will be available, what features will be in the release, and how much will it cost. See also *fixed-date release, fixed-scope release*.

release planning. Longer-term planning that answers questions like "When will we be done?" or "Which features can I get by the end of the year?" or "How much will this cost?" Release planning must balance customer value and overall quality against the constraints of scope, schedule, and budget. See also *release plan*.

release train. An approach to aligning the vision, planning, and interdependencies of many teams by providing cross-team synchronization based on a common cadence. A release train focuses on fast, flexible flow at the level of a larger product. See also *scrum of scrums*.

retrospective. See *sprint retrospective*.

risk. 1. The likelihood that an event will be accompanied by undesirable consequences. Risk is measured by both the probability of the event and the seriousness of the consequences. 2. Any uncertainty that is expected to have a negative outcome for the activity. See also *uncertainty.*

role. A cohesive set of responsibilities that may be fulfilled by one or more people. The three Scrum roles are product owner, ScrumMaster, and development team. See also *practice, principle*.

rule. A common practice or generally reliable method of action in a particular situation. A rule may be broken when the pragmatics of a situation dictate that a different course of action should be pursued. The Scrum framework includes rules. See also *essential Scrum, Scrum framework*.

S

Scrum. A term borrowed from the sport of rugby. 1. A lightweight agile framework for managing complex product and service development. 2. An iterative and incremental approach to developing products and managing work. See also *agile, Scrum framework*.

Scrum framework. A collection of values, principles, practices, and rules that form the foundation of Scrum-based development. See also *Scrum*.

ScrumMaster. The coach, facilitator, impediment remover, and servant leader of the Scrum team. The ScrumMaster is one of the three roles on a Scrum team. The ScrumMaster provides process leadership and helps the Scrum team and the rest of the organization develop their own high-performance, organization-specific Scrum approach. See also *Scrum team, servant leader.*

Scrummerfall. See *WaterScrum*.

scrum of scrums (SoS). An approach to coordinating the work of multiple Scrum teams wherein one or more members of each Scrum team come together to discuss and resolve inter-team dependency issues. See also *release train*.

Scrum team. A team composed of a product owner, ScrumMaster, and development team that works on a Scrum development effort. See also *development team, product owner, ScrumMaster.*

self-organization. 1. A bottom-up emergent property of a complex adaptive system whereby the organization of the system emerges over time as a response to its environment. 2. A property of a development team that organizes itself over time, without an external dominating force applying traditional top-down, command-and-control management. 3. Reflects the management philosophy whereby operational decisions are delegated as much as possible to those who have the most detailed knowledge of the consequences and practicalities associated with those decisions. See also *complex adaptive system, emergence.*

sequential process. See *plan-driven process.*

servant leader. 1. A person who achieves results for her organization by giving priority attention to the needs of her colleagues and those she serves. 2. A philosophy and practice of leadership based on listening, empathy, healing, awareness, persuasion, conceptualization, foresight, stewardship, commitment, and community building. See also *ScrumMaster.*

silent grouping. A facilitation technique for getting people to group related items without talking, relying only on the individual placement and movement of items (typically cards or sticky notes) as a means of communicating and coordinating among the participants. A technique frequently used during the sprint retrospective activity. See also *sprint retrospective.*

simple domain. 1. A situation in which everyone can see cause and effect. Often the right answer is obvious and undisputed. 2. One of the domains in the Cynefin framework. See also *Cynefin.* Contrast with *chaotic domain, complex domain, complicated domain, disorder domain.*

single-piece flow. A state where items are produced one at a time and flow (are pulled) through the development process as a single unit.

solution. A product or a service that results from a development effort.

SoS. See *scrum of scrums.*

specification by example. See *acceptance-test-driven development.*

sprint. A short-duration, timeboxed iteration. Typically a timebox between one week and a calendar month during which the Scrum team is focused on producing a potentially shippable product increment that meets the Scrum team's agreed-upon definition of done. See also *definition of done, iteration, potentially shippable product increment.*

sprintable story. See *implementable story.*

sprint backlog. 1. An artifact produced at a sprint-planning meeting and continuously updated during sprint execution that helps a self-organizing team better plan and manage the work necessary to deliver on the sprint goal. 2. A list of the product backlog items pulled into a sprint and an associated plan for how to achieve them—frequently expressed in terms of tasks that are estimated in ideal hours. See also *ideal hour, sprint planning, task.*

sprint demo. 1. An activity of a sprint review where the completed (done) product backlog items are demonstrated with the goal of promoting an information-rich discussion between the Scrum team and other sprint review participants. 2. A term that is frequently used synonymously to refer to the entire sprint review. See also *sprint review.*

sprint goal. A high-level summary of the goal the product owner would like to accomplish during the sprint. Frequently elaborated through a specific set of product backlog items.

sprint planning. A time when the Scrum team gathers to agree on a sprint goal and determine what subset of the product backlog it can deliver during the forthcoming sprint. During sprint planning, a sprint backlog is produced to help the team acquire confidence that it can deliver the committed product backlog items. See also *sprint backlog, sprint goal.*

sprint retrospective. An inspect-and-adapt activity performed at the end of every sprint. The sprint retrospective is a continuous improvement opportunity for a Scrum team to review its process (approaches to performing Scrum) and to identify opportunities to improve it. See also *inspect and adapt, sprint retrospective.*

sprint review. An inspect-and-adapt activity that occurs after sprint execution where the Scrum team shows to all interested parties what was accomplished during the sprint. The sprint review gives everyone with input in the product development effort an opportunity to inspect what has been built so far and adapt what will be built next. See also *inspect and adapt, sprint demo.*

stakeholder. A person, group, or organization that affects or can be affected by an organization's actions. See also *external stakeholders, internal stakeholders.*

stakeholder value. The value that a solution delivers to stakeholders. Sometimes used interchangeably with customer value. See also *stakeholder.*

story. See *user story.*

story mapping. 1. A technique that takes a user-centric perspective for generating a set of user stories. Each high-level user activity is decomposed into a workflow that can be further decomposed into a set of detailed tasks. 2. A two-dimensional representation of a traditional one-dimensional product backlog list. See also *product backlog, user story.*

story point. A measure of the relative size of product backlog items that takes into account factors such as complexity and physical size. Typically determined by engaging in Planning Poker. See also *ideal day, Planning Poker, relative size measure.*

strategic filter. The decision criteria used by an organization to evaluate whether a proposed product meets the strategic criteria to move forward for additional consideration. Contrast with *economic filter.*

strategic technical debt. A form of technical debt that is used as a tool to help organizations better quantify and leverage the economics of important, often time-sensitive, decisions. Sometimes taking on technical debt for strategic reasons is a sensible business choice. Contrast with *naive technical debt, unavoidable technical debt.* See also *technical debt.*

sustainable pace. The appropriately aggressive pace at which a team works so that it produces a good flow of business value over an extended period of time without getting burned out.

swarming. A behavior whereby team members with available capacity and appropriate skills collectively work (swarm) on an item to finish what has already been started before moving ahead to begin work on new items. See also *T-shaped skills.*

synchronization. Causing multiple events to happen at the same time. Frequently used to ensure that multiple Scrum teams work together in a coordinated way by starting and ending their sprints on the same days. See also *cadence.*

T

tacit knowledge. Unwritten and unspoken knowledge (including insights, intuitions, and hunches) that is hard, but not impossible, to articulate with formal language. The opposite of explicit or formal knowledge. Sometimes referred to as "know-how."

targeted technical debt. A status category for technical debt that represents debt that is known and has been targeted for servicing by the development team. Contrast with *happened-upon technical debt, known technical debt.* See also *technical debt.*

task. The technical work that a development team performs in order to complete a product backlog item. Most tasks are defined to be small, representing no more than a few hours to a day or so of work.

task board. An information radiator used during sprint execution to communicate the progress and flow of task-level work within a sprint. See also *information radiator, task.*

TDD. See *test-driven development.*

team. A small, cross-functional collection of diverse, collaborating people who are aligned to a common purpose and goal. Team members trust each other and work together to achieve the goal, holding themselves mutually accountable for the outcome. Contrast with *group.*

technical debt. 1. A term used to describe the obligation that a software organization incurs when it chooses a design or construction approach that is expedient in the short term but that increases complexity and is more costly in the long term. 2. A metaphor that facilitates the communication between business and technical people regarding implementation artifact inadequacies. See also *naive technical debt, strategic technical debt, unavoidable technical debt.*

technical practices. The specific practices or techniques that are used during sprint execution to properly perform the work required to deliver features that have manageable levels of technical debt and meet the Scrum team's definition of done.

technical stories. A "user" story (product backlog item) that delivers no perceived end-user value but does deliver important architecture or infrastructure needed to deliver future user value. See also *user story.*

technique. A defined procedure that is used to perform some or all of an activity or support an approach. See also *activity, approach.*

test-driven development (TDD). 1. An evolutionary approach to development based on writing a failing automated test before the functional code that makes the test pass. Once the code is written to pass the test, the cycle is then repeated, including refactoring the existing code to ensure a coherent cross-functional design. The goal of test-driven development is specification and not validation—to think through a

design before code is written, to create clean code that always works. 2. An example of test-first development. See also *refactoring, technical practices, test-first development*.

test-first development. A technical practice where the tests are written before the development is performed. An example is test-driven development. See also *technical practices, test-driven development*.

theme. A collection of related user stories. A theme provides a convenient way to indicate that a set of stories have something in common, such as being in the same functional area. See also *epic, user story*.

timebox. A fixed-length period of time during which an activity is performed. In Scrum, sprints are timeboxed iterations where a team works at a sustainable pace to complete a chosen, WIP-limited set of work. See also *sprint, timeboxing*.

timeboxing. A time management technique that helps organize the performance of work and manage scope. See also *timebox*.

traditional development process. See *plan-driven process*.

transparency. One of the three pillars of empirical process control; open access to the unbiased information required for inspection and adaptation. See also *adaptation, empirical process control, inspection*.

T-shaped skills. A metaphor used to describe a person with deep vertical skills in a specialized area (such as UX design) as well as broad but not necessarily very deep skills in other relevant areas (such as testing and documentation). Team members with T-shaped skills better enable swarming behavior. See also *swarming*.

U

unavoidable technical debt. A form of technical debt that is usually unpredictable and unpreventable and accrues through no fault of the team building the product. Contrast with *naive technical debt, strategic technical debt*. See also *technical debt*.

uncertainty. Something that is not known or established. Often considered synonymous with risk but is actually broader in scope because uncertainty includes both risks (negative outcomes) and opportunities (positive outcomes). See also *risk*.

unknown unknowns. The things that we don't yet know that we don't know.

unnecessary formality. 1. A ceremony that has a real cost but delivers little or no value (a form of waste). 2. Process for the sake of process. See also *ceremony, waste*.

user role. 1. The name for a class of product users. 2. One of the key elements of a user story that defines the recipient of the value delivered by a user story. See also *user story*.

user story. A convenient format for expressing the desired business value for many types of product backlog items. User stories are crafted in a way that makes them understandable for both business people and technical people. They are structurally simple and typically expressed in a format such as "As a <user role> I want to achieve <goal> so that I get <benefit>." They provide a great placeholder for a conversation. Additionally, they can be written at various levels of granularity and are easy to progressively refine. See also *epic, progressive refinement, theme, user role.*

user-story-writing workshop. A workshop lasting from a few hours to a few days where a diverse team of participants collectively brainstorms desired business value and creates user story placeholders for what the product or service is supposed to do. See also *user story.*

V

validated learning. A term proposed by Ries (2011) to describe the progress made when important assumptions have been confirmed or refuted by subjecting each assumption to one or more customer validation tests. Contrast with *assumption.*

values. 1. Those things that we hold dear or precious. 2. The foundation of a shared operating agreement among members of a team. Core Scrum values include honesty, openness, courage, respect, focus, trust, empowerment, and collaboration.

variability. The spread or dispersion of a set of data representing non-identical outcomes. In manufacturing, variability is always waste. In product development, some variability is necessary to develop innovative solutions. See also *waste.*

velocity. A measure of the rate at which work is completed per unit of time. Using Scrum, velocity is typically measured as the sum of the size estimates of the product backlog items that are completed in a sprint. Velocity is reported in the same units as product backlog items—usually story points or ideal days. Velocity measures output (the size of what was delivered), not outcome (the value of what was delivered).

W

waste. Any activity that consumes resources and produces no added value to the product or service that a customer receives.

waterfall. A term referring to the graphical depiction of a development process in which the sequential phases of work are shown flowing steadily downwards like a cascading waterfall. See also *plan-driven process.*

waterfall process. See *plan-driven process.*

WaterScrum. Overlaying waterfall-style development on the Scrum framework. An example would be performing an analysis sprint, followed by a design sprint, followed by a coding sprint, followed by a testing sprint. Synonymous with *Scrummerfall*.

weighted shortest job first (WSJF). An economically optimal algorithm for scheduling work in an environment where both the cost of delay and the duration vary among the work items. See also *cost of delay*.

WIP. See *work in process*.

won't-have features. The set of features that are specifically declared to not be in the upcoming release. Contrast with *must-have features, nice-to-have features*.

work in process (WIP). Work that has entered the development process but is not yet finished and available to a customer or user. Refers to all assets or work products of a product or service that are currently being worked on or waiting in a queue to be worked on.

WSJF. See *weighted shortest job first*.

X

XP. See *Extreme Programming*.

References

Adkins, Lyssa. 2010. *Coaching Agile Teams: A Companion for ScrumMasters, Agile Coaches, and Project Managers in Transition.* Addison-Wesley Professional.

Anderson, David J. 2010. *Kanban*. Blue Hole Press.

Appelo, Jurgen. 2011. *Management 3.0: Leading Agile Developers, Developing Agile Leaders*. Addison-Wesley Professional.

Beck, Kent, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, and Dave Thomas. 2001. *Manifesto for Agile Software Development.* www.agilemanifesto.org/.

Beck, Kent, and Cynthia Andres. 2004. *Extreme Programming Explained*, 2nd ed. Addison-Wesley Professional.

Boehm, Barry W. 1981. *Software Engineering Economics*. Prentice Hall.

Brooks, Frederick P. 1995. *The Mythical Man-Month: Essays on Software Engineering*, 2nd ed. Addison-Wesley Professional. (Originally published in 1975.)

Cohn, Mike. 2004. *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional.

———. 2006. *Agile Estimating and Planning.* Addison-Wesley Professional.

———. 2009. *Succeeding with Agile*. Addison-Wesley Professional.

———. 2010. Agile 2010 keynote presentation.

Cook, Daniel. 2008. "The Laws of Productivity: 8 productivity experiments you don't need to repeat." Presentation found at http://www.lostgarden.com/2008/09/rules-of-productivity-presentation.html.

Crispin, Lisa, and Janet Gregory. 2009. *Agile Testing: A Practical Guide for Testers and Agile Teams*. Addison-Wesley Professional.

Cunningham, Ward. 1992. "The WyCash Portfolio Management System," OOPSLA 1992 experience report. OOPSLA '92, Object-Oriented Programming Systems, Languages and Applications, Vancouver, BC, Canada, October 18–22.

Denne, Mark, and Jane Cleland-Huang. 2003. *Software by Numbers: Low-Risk, High-Return Development*. Prentice Hall.

Derby, Esther, and Diana Larsen. 2006. *Agile Retrospectives: Making Good Teams Great*. Pragmatic Bookshelf.

Fowler, Martin. 2009. "Technical Debt Quadrant." Bliki entry found at http://martinfowler.com/bliki/TechnicalDebtQuadrant.html.

Fowler, Martin, Kent Beck, John Brant, William Opdyke, and Don Roberts. 1999. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley Professional.

Goldberg, Adele, and Kenneth S. Rubin. 1995. *Succeeding with Objects: Decision Frameworks for Project Management*. Addison-Wesley Professional.

Grenning, James. 2002. "Planning Poker." www.objectmentor.com/resources/articles/PlanningPoker.zip.

Highsmith, Jim. 2009. *Agile Project Management: Creating Innovative Products,* 2nd ed. Addison-Wesley Professional.

Hohmann, Luke. 2003. *Beyond Software Architecture*. Addison-Wesley Professional.

IEEE. 1990. IEEE Std 610.12-1990 (revision and designation of IEEE Std 792-1983). IEEE Standards Board of the Institute of Electrical and Electronics Engineers, New York, September 28, 1990.

Jeffries, Ron. 2001. "Essential XP: Card, Conversation, Confirmation." http://xprogramming.com/articles/expcardconversationconfirmation/.

Katz, Ralph. 1982. "The Effects of Group Longevity on Project Communication and Performance." *Administrative Science Quarterly* 27: 81–104.

Kennedy, John Fitzgerald. 1961. Special Message to the Congress on Urgent National Needs, May 22.

Kerth, Norm. 2001. *Project Retrospectives: A Handbook for Team Reviews*. Dorset House.

Larman, Craig, and Bas Vodde. 2009. "Lean Primer." Downloadable from www.leanprimer.com/downloads/lean_primer.pdf.

Laufer, Alexander. 1996. *Simultaneous Management: Managing Projects in a Dynamic Environment*. American Management Association.

Leffingwell, Dean. 2011. *Agile Software Requirements: Lean Requirements Practices for Teams, Programs, and the Enterprise*. Addison-Wesley Professional.

McConnell, Steve. 2007. "Technical Debt." Blog entry found at http://blogs.construx.com/blogs/stevemcc/archive/2007/11/01/technical-debt-2.aspx.

Mar, Kane. 2006. "Technical Debt and the Death of Design: Part 1." Blog entry found at http://kanemar.com/2006/07/23/technical-debt-and-the-death-of-design-part-1/.

Martin, Robert C. 2008. *Clean Code: A Handbook of Agile Software Craftsmanship.* Prentice Hall.

Page, Scott. 2007. *The Difference: How the Power of Diversity Creates Better Groups, Firms, Schools, and Societies.* Princeton University Press.

Patton, Jeff. 2008. Example of incremental releasing. Personal communication.

———. 2009. "Telling Better User Stories: Mapping the Path to Success." *Better Software*, November/December, 24–29.

Pelrine, Joseph. 2011. "Is Software Development Complex." Guest blog entry found at http://cognitive-edge.com/blog/entry/4597/is-software-development-complex.

Pichler, Roman. 2010. *Agile Product Management with Scrum: Creating Products That Customers Love.* Addison-Wesley Professional.

PMI. 2008. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide),* 4th ed. Project Management Institute, Inc.

Poppendieck, Mary, and Tom Poppendieck. 2003. *Lean Software Development: An Agile Toolkit.* Addison-Wesley Professional.

Putnam, Doug. 1996. "Team Size Can Be the Key to a Successful Project." An article in QSM's Process Improvement Series. www.qsm.com/process_01.html.

Putnam, Lawrence H., and Ware Myers. 1998. "Familiar Metrics Management: Small Is Beautiful—Once Again." *IT Metrics Strategies* IV:8: 12–16. Cutter Information Corp.

Reinertsen, Donald G. 2009a. "Types of Processes." Guest blog entry found at www.netobjectives.com/blogs/Types-of-Processes.

———. 2009b. *The Principles of Product Development Flow: Second Generation Lean Product Development.* Celeritas Publishing.

Ries, Eric. 2011. *The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses.* Crown Business.

Schwaber, Ken. 1995. "Scrum Development Process." In *OOPSLA Business Object Design and Implementation Workshop*, ed. J. Sutherland et al. Springer.

———. 2004. *Agile Software Development with Scrum.* Microsoft Press.

Schwaber, Ken, and Mike Beedle. 2001. *Agile Software Development with Scrum.* Prentice Hall.

Schwaber, Ken, and Jeff Sutherland. 2011. "The Scrum Guide." Downloadable at www.scrum.org.

SEI. 2010. "CMMI for Development, Version 1.3." Software Engineering Institute, Carnegie Mellon University. Downloadable at www.sei.cmu.edu/library/abstracts/reports/10tr033.cfm.

———. 2011. Second International Workshop on Managing Technical Debt, May 23. Colocated with ICSE 2011, Waikiki, Honolulu, Hawaii. Downloadable at www.sei.cmu.edu/community/td2011/.

Smith, Preston G., and Donald G. Reinertsen. 1998. *Developing Products in Half the Time: New Rules, New Tools*. Van Nostrand Reinhold.

Snowden, David J., and Mary E. Boone. 2007. "A Leader's Framework for Decision Making." *Harvard Business Review*, November.

Staats, Bradley R. 2011. *Unpacking Team Familiarity: The Effects of Geographic Location and Hierarchical Role.* University of North Carolina at Chapel Hill.

Takeuchi, Hirotaka, and Ikujiro Nonaka. 1986. "The New New Product Development Game." *Harvard Business Review*, January, 137–146.

Tuckman, Bruce W. 1965. "Developmental Sequence in Small Groups." *Psychological Bulletin* 63: 384–399. The article was reprinted in *Group Facilitation: A Research and Applications Journal*, no. 3, Spring 2001.

VersionOne. 2011. "The State of Agile Development: Sixth Annual Survey." Posted as a downloadable PDF in the Library of White Papers on www.versionone.com.

Wake, William C. 2003. "INVEST in Good Stories, and SMART Tasks." www.xp123.com.

Wheelwright, Steven C., and Kim B. Clark. 1992. *Revolutionizing Product Development: Quantum Leaps in Speed, Efficiency, and Quality.* The Free Press.

Wiseman, John "Lofty." 2010. *SAS Survival Guide: For Any Climate, in Any Situation*, rev. ed. Collins Reference.

INDEX

A

Absolute sizes, vs. relative sizes in estimation, 125–128
Acceptance criteria
  conditions of satisfaction related to product backlog, 77
  defined, 401
  definition of ready and, 110
  product owner defining and verifying, 169–170
  user stories containing confirmation information, 85–86
Acceptance-test-driven development (ATTD), 85–86, 402
Acceptance tests
  conditions of satisfaction expressed via, 85
  defined, 401
  product owner responsibilities and, 169
  verifying conditions of satisfaction, 77
Accountability, of product owner, 173
Accrual of technical debt, managing, 149–152
Accuracy
  defined, 402
  vs. precision in estimation, 125, 274–275
Actions, resulting from retrospective
  deciding what action to take, 389–390
  determining possible actions, 387–388
  follow through on, 391–392
  as output of sprint retrospective, 381
  selecting insights to act on, 388
Activities
  defined, 402
  overview of, 16–18
Adaptation. *See also* Prediction and adaptation principle, in agile development
  balancing predictive work with adaptive work, 43–44
  based on product review, 371
  daily scrum as inspect-and-adapt activity, 354
  defined, 402
  discovering your own path forward, 396
  and exploration in approach to development, 39–40
  as focus of planning rather than conformance, 249–251
  leveraging variability, 35–36
  plan-driven development compared with agile development, 59
  responsibilities of development team, 197–198
  sprint retrospective and, 375
  sprint review and, 363
Agile development
  concerns about adopting, 225
  defined, 402
  managers promoting agile values, 233–234
  no end state in, 395
  overview of, 1–3
  plan-driven approach compared with, 59–60
  product backlog in, 1
  sharing best practices, 396–397
The Agile Manifesto (Beck), xxxi, 30, 204–205, 210
Agile principles
  accepting that you can't get it right up front, 38–39
  adapting to real-time information and replanning based on, 54
  adaptive, exploratory approach, 39–40
  balancing predictive work with adaptive work, 43–44
  batch sizes in, 48–49
  cost of change and, 40–43
  cost of delays and, 52–54

Agile principles (*continued*)
embracing helpful variability, 32–33
focusing on idle work, 51–52
inspection, adaptation, and transparency, 35–36
inventory management, 49–50
iterative and incremental approach to development, 33–35
keeping options open, 37–38
learning loops in, 45–46
measuring progress by asset validation, 54–55
minimizing unnecessary formality, 57–58
organizing workflow for fast feedback, 46–47
overview of, 29–32
prediction and adaptation, 37
quality built-in to development process, 56–57
reducing uncertainty, 36–37
sustainable pace in performance of work, 56
validated learning in, 44–45
value-centric delivery in, 55
variability and uncertainty and, 32
work in process (WIP) and, 48
Agile Retrospectives (Derby and Larsen), 379
All-at-once product development
defined, 402
in origins of Scrum, 3
All-before-any approach
defined, 402
to work in process, 48
Anticipatory process. *See* plan-driven development
Appelo, Jurgen, 230
Approach
defined, 402
essential scrum includes, xxix
realizing Scrum practices, 13
Artifacts
defined, 402
just-in-time approach to creating work products, 43
managing inventory of planning artifacts, 251–252
potentially shippable product increment as, 25–26
product backlog as, 18–19
sprint backlog as, 18
Asset teams, 214. *See* also Component teams
Assets
measuring progress by asset validation, 54–55
monitoring and reports focusing on asset validation, 236
Assumptions
calculating release costs and, 325–326
defined, 402
replanning based on validation of, 251
validated learning and, 45, 304
Atmosphere, setting for sprint retrospective, 382
ATTD (Acceptance-test-driven development), 85–86, 402
Attendance
sprint retrospective issues, 392
sprint review issues, 372–373
Authority, levels of (Appelo), 230
Automated testing, 149, 355–356

B

Batch size
in agile development, 48–49
comparing plan-driven development with agile development, 60
defined, 403
Benefits of Scrum, 4–5
Best practices, 396–397
Blame
creating blame-free atmosphere for sprint retrospective, 382
sprint retrospective issues, 393
“Boil-the-ocean” projects, 65
Boy Scout rule
defined, 403
servicing technical debt when you happen upon it, 158–159
Budget constraint
fixed date approach, 313–314
fixed everything approach, 311–312
fixed scope and date approach, 312–313
fixed scope approach, 313
in release planning, 311

Burndown chart
defined, 403
for fixed-scope release, 327–328
sprint, 357–359
Burnup charts
defined, 403
for fixed-scope release, 328–329
sprint, 359–360
Business
engagement pattern with, 170
making technical debt visible at business level, 153–154
ScrumMaster skills related to business domain, 188

C

Cadence
benefits of consistent duration of sprints, 67–68
defined, 403
Capacity
defined, 403
in Kanban, 10
measuring in effort-hours, 342–343
measuring in story points, 342
sprint planning, 22, 340–342
underutilization of, 351
Card format, for user stories, 83–84
Ceremony
defined, 403
minimizing unnecessary, 57–58, 368
plan-driven development compared with agile development, 60
Change
consequences of, 70–71
handling cost of, 40–43
maintaining sprint goals despite, 69–73
managing, 79
overcoming the status quo, 398–399
as product backlog item, 101
Change agent, ScrumMaster as, 187, 191
Chaotic domain
in Cynefin framework, 6–7, 9
defined, 403
Checkpoints, short duration sprints providing frequent, 66–67
Chickens and pigs, 25, 403
Chief product owner, 183–184, 404
Clarification, of sprint goals, 69–70
Closing retrospectives, 390–391
Closure, timeboxing enforcing, 63
CMMI maturity model, 395
Coach
a day in the life of ScrumMaster, 190
ScrumMaster as, 16, 185–186
Code refactoring. *See* Refactoring code
Cohn, Mike, xxv, xxxiii–xxxiv, 129–130, 206, 395, 397–398
Collaboration
benefits of face-to-face communication, 205
cross-cluster, 240–241
funneling through project manager, 242–243
of product owner with development team, 170–171
of product owner with stakeholders, 171
ScrumMaster skills, 189
in sprint review, 370
Commercial development projects, 177–179
Commercial-off-the-shelf (COTS), 8
Commitment
as basis of sprint goals, 69
change and, 71
checking if realistic, 344–345
defined, 404
of development team, 207–208
estimates contrasted with, 124–125
sporadic attendance and, 372–373
sprint planning outcomes and, 17–18, 346
Communication
channels between teams, 240–241
development team skills/characteristics, 204–205
facilitating shared understanding, 81–82
product owner skills, 172–173
of progress in fixed-date release, 329–330
of progress in fixed-date release planning, 327–329
ScrumMaster skills, 189–190
of sprint execution progress, 356
transparency of, 205–206
Competence, managers developing team members, 231–232

Complaints, sprint retrospective issues, 393
Complex adaptive systems
  defined, 404
  flight pattern of geese illustrating, 199
  Scrum origins and, 3
Complex domain
  in Cynefin framework, 6–8
  defined, 404
Complexity, Scrum providing confidence in handling, 6
Complicated domain
  in Cynefin framework, 6–8
  defined, 404
Component teams
  combining with feature teams, 217–218
  defined, 404
  feature teams compared with, 213–216
  product owner for, 177, 180–181
  when defining a product, 116
  when to use, 216
Components
  development of, 213–214
  development projects, 180–181
  integration and testing, 46–47
Conditions of satisfaction, 404. *See also* Acceptance criteria
Confidence, acquiring in sprint planning, 344–346
Confidence threshold
  defined, 404
  for product planning, 290, 298
  targeting realistic, 300–302
Confirmation information, in user stories, 85–86
Conformance to plans, in plan-driven development, 54, 60, 250
Constraints, in release planning
  fixed date, 313–314
  fixed everything approach, 311–312
  fixed scope, 313
  fixed scope and date, 312–313
  inputs to sprint planning, 338
  overview of, 311
  updating, 314
  variable quality and, 313–314
Continuous deployment (delivery)
  defined, 404
  planning release of features to customers, 308
  product roadmap and, 260
Continuous improvement
  no end state in Scrum, 395
  sprint retrospective and, 375
  while applying iterative and incremental development, 34
Continuous integration
  defined, 404
  helping work at a sustainable pace, 209
  technical practice, 355
  use of good practices prevents accrual of technical debt, 149
Contracts, limitations of fixed-price contracts, 235
Conversations
  in development of user stories, 84–85
  facilitating shared understanding, 81–82
Coordination. *See also* Collaboration
  cross-cluster, 240–241
  funneling through project manager, 242–243
Cost of delay
  Agile principles and, 52–54
  comparing plan-driven development with agile development, 60
  defined, 404
  portfolio planning and, 271–274
  for properly quantifying technical debt economics, 150–152
Costs
  calculating in release planning process, 325–326
  handling cost of change, 40–43
  of idle work, 52–53
  Scrum reducing, 6
  technical debt impacting development costs, 142–143
COTS (Commercial-off-the-shelf), 8
Cross-cluster collaboration, 240–241
Cross-functional diversity and sufficiency, of development team, 200–201
Cross-functional teams
  in agile development, 2
  defined, 405
  feature teams, 213

high-bandwidth communication and, 205
managers forming, 228–229
quality built-in to development process, 56–57
vs. role-specific teams, 195–196
Cunningham, Ward
on refactoring, 149
on technical debt, 139–140
Customer satisfaction
Scrum benefits, 6
technical debt decreasing, 144
Customer uncertainty
defined, 405
reducing, 36
Customers
engagement pattern with, 170–171
planning release of features to, 307
product owner understanding needs of, 166
repaying technical debt while performing customer-valuable work, 160–162
value-centric delivery focused on needs of, 55
value of user stories to, 90
Cynefin framework
defined, 405
for situation-appropriate decision making, 6–7

D

Daily planning, 258, 264–265
Daily scrum
approaches to, 397
daily planning during, 264–265
defined, 405
sprint execution and, 354
sprints and, 23–25
when grooming occurs and, 108
Daily stand-up. *See* Daily scrum
Date constraint
fixed date approach, 313–314
fixed everything approach, 311–312
fixed scope and date approach, 312–313
fixed scope approach, 313
in release planning, 311
Deadlines, resulting in technical debt, 144–145
Decision making
economic filter for go/no-go decision making, 275–276
illusion of certainty and, 303
incremental/provisional approach to funding, 304–305
keeping options open, 37–38, 249
plan-driven development compared with agile development, 59
by product owner, 173
which work needs to be done, 353–354
which work to start, 352
DEEP (Detailed appropriately, Emergent, Estimated, and Prioritized)
appropriate detail, 101–102
characteristics of good product backlog, 101
defined, 405
emergent nature of, 102
prioritization in, 103–104
size estimates in, 102–103
Defects
compounding, 142
defining when sprint is complete or done and, 75
as product backlog item, 100–101
as technical debt, 139
Defined process, in plan-driven development, 32
Definition of done
checklist, 74–76
confidence threshold as envisioning, 290
defined, 405
development team needs skills to meet, 200–201
evolving over time, 76–77
for managing technical debt, 149–150
no end state in Scrum, 395
nonfunctional requirements for inclusion in, 93
overview of, 25–26
preventing accrual of technical debt, 150
versus acceptance criteria, 77
what work needs to be done, 353–354
Definition of ready
acceptance criteria, 169
checklist, 109–110
defined, 405

Definition of ready (*continued*)
overview, 108–110
product backlog items for sprint planning, 336
providing boundaries for work at the task level, 353
selecting product backlog items and, 344
understanding how to demonstrate items at sprint review, 370
Delays, cost of. *See* Cost of delay
Delegation, as means of empowering teams, 230
Deliverables. *See also* Potentially shippable product increments (PSIs)
short duration sprints and, 65–66
technical debt increasing time to delivery, 142
Demonstration aspect, of sprint review, 368, 370
Design flaws, as technical debt, 139
Detail
in product backlog, 101–102
in user stories, 86–88
Development team
accountability of product owner to, 173
communication skills of, 204–205
cross-functional diversity and sufficiency of, 200–201
daily scrum for, 23
defined, 405
focus and commitment of, 207–208
grooming product backlog, 105–106
long-lived nature of, 209–211
multiple teams with one product backlog, 115–116
Musketeer attitude (all for one, one for all), 203–204
one team with multiple product backlogs, 117–118
overview of, 195
participant in product planning, 288–289
participant in release planning, 308
participant in requirements conversation, 84
participant in sprint execution, 348
participant in sprint planning, 335
participant in sprint retrospective, 377
participant in sprint review, 364–365
PBI estimation, 123–124
product owner collaborating with, 166, 170–171
responsibilities of, 196–197
role of, 16
role-specific teams, 195–196
rules of Planning Poker and, 132
as Scrum role, 16
self-organizing nature of, 198–200
small size of, 206
sprint planning, 21–23
sustainable pace in performance of work, 208–209
T-shaped skills, 201–203
technical practices for task performance, 355–356
transparency of communication, 205–206
when grooming occurs and, 107–108
Discussions
in sprint review, 371
when estimating, 121–122
when writing user stories, 81–82, 85
Disorder domain
in Cynefin framework, 6–7, 9
defined, 406
Diversity, of development team, 200–201
Documentation
conversations compared with, 81–82
cost of delay example involving, 53–54
in definition of done, 74
lack of in VersionOne 2011 survey, 225
plan-driven development as document-centric process, 57
in Scrum development, 57–58
supplementing user stories, 84
Domain skills, of product owner, 171–172
Done
acceptance criteria compared with, 77
checklist, 74–76
defined, 406
done vs. done done, 77–78
evolution of definition of done over time, 76–77
no end state in Scrum, 395
sprint review confirming, 367–368
value of strong definition of done in preventing accrual of technical debt, 149–150

Dot voting
defined, 406
selecting which insights to act on, 388
Duration, of sprints
calculating from estimated size and measured velocity, 119–120
consistency of, 67–68
short duration preference, 64–67
story points in calculation of, 128

E

Economic filter
defined, 406
for go/no-go decision making, 275–276
Economics
of abnormal sprint termination, 72–73
of aligning all teams to a single product backlog, 115–116
of change, 70–71
of component teams, 218
of developing a project plan per sprint, 349
focusing on short time horizon in product planning, 302
improved by fast feedback, 65
incremental/provisional funding, 304–305
learning fast and pivoting as necessary, 305
of long-lived teams, 210
managing, 167–168, 236
marginal economics applied to in-process products, 283–285
of product planning, 299–300
of release approach, 253
of rising development costs, 142
single versus multiple release, 252–253
of smaller, more frequent releases, 279–280
speed and efficiency and, 302–303
targeting realistic confidence threshold, 300–302
of technical debt, 150–152
validated learning, 303–304
Economies of scale, manufacturing vs. product development, 48
Effort/cost, scheduling portfolio backlog items and, 274
Effort hours
capacity in, 342–343
checking if commitment is realistic, 344–345
tasks in, 122
Emergent opportunities
defined, 406
embracing quickly, 278–279
Emotions seismograph
defined, 406
mining for insights, 385
sprint retrospective and, 384–385
Empirical process control (Schwaber and Beedle), 35, 406
End of life, not repaying technical debt for products approaching, 157
End uncertainty
defined, 406
reducing uncertainty, 36
Enjoyment, Scrum benefits, 6
Enterprise Transition Community (ETC), 398
Environment, managers' responsibilities
aligning internal groups, 234
aligning partners, 234–235
promoting agile values, 233–234
removing organizational impediments, 234
Envisioning. *See* Product planning
Epics
defined, 406
estimating, 103
in release train, 221
representing product backlog items, 294–295
size of user stories and, 86–88
story mapping technique and, 96
Errors, setting limit or bound on, 65
Estimable criteria, INVEST, 91–92
Estimation
accuracy vs. precision, 125, 274–275
commitments contrasted with, 124–125
defined, 407
development team in PBI estimation, 123–124
ideal days for measurements in, 128–129
overview of, 119–120
of PBIs, 121
Planning Poker approach, 129–133
of product backlog, 121–122

Estimation (*continued*)
product owner and, 175
relative sizes vs. absolute sizes, 125–128
scale in, 130
story points for measurements in, 128
of tasks, 122
units for, 128
what and when of, 120–121
ETC (Enterprise Transition Community), 398
Event timeline
defined, 407
mining for insights, 385
sprint retrospective and, 384
Excitement/enthusiasm, short duration sprints rejuvenating, 65–66
Exercises
inputs for sprint retrospective, 380
selecting for use in sprint retrospective, 379
Experiments, knowledge-acquisition user stories, 93
Exploitation, 59, 407
Exploration
defined, 407
knowledge-acquisition user stories, 93–95
plan-driven development compared with agile development, 39–40, 59
External stakeholders
defined, 407
product owner collaborating with, 171
Extreme Programming (Beck and Andres), 355, 407

F

Face-to-face communication, 205
Facilitator
ScrumMaster as, 16
for sprint execution, 348
for sprint planning, 335–336
for sprint retrospective, 393
for sprint review, 368
Fail fast, 305, 407
Fast feedback. *See also* Feedback
defined, 407
early review and, 367
monitoring and reports aligned to, 236
organizing workflow for, 46–47
plan-driven development compared with agile development, 59
short duration sprints aiding, 64–65
technical debt and, 139
Fast pace, of development
fast delivery as Scrum benefit, 6
go fast but never hurry, 56
Feature teams
combining with component teams, 217–218
comparing with component teams, 213–216
defined, 407
product owner, 180-181
Features
defined, 407
as product backlog item, 100–101
release flow management and, 110–111
user stories and, 87–88
Feedback. *See also* Fast feedback
in iterative and incremental development, 34–35
learning fast and pivoting as necessary, 305
learning loops and, 45–46
organizing workflow for, 46–47
performance feedback given by managers, 232
plan-driven development compared with agile development, 59
in prioritization of iterations, 2
short duration sprints aiding, 64–65
sprint review and, 364–365
Fixed date constraint, in release planning, 313–314
Fixed-date release
calculating costs in, 325–326
communicating progress of, 329–330
defined, 408
overview of, 318–323
planning and, 67–68
Fixed everything constraint, in release planning, 311–312
Fixed-price contracts, limitations of, 235
Fixed scope and date constraints, in release planning, 312–313
Fixed scope constraint, in release planning, 313

Fixed-scope release
calculating costs in, 325–326
communicating progress of, 327–329
defined, 408
planning, 323–325
product roadmap and, 260–261
Flow
daily scrum in management of, 354
deciding which work needs to be done, 353–354
deciding which work to start, 352
defined, 408
managing in sprint execution, 349–350
organizing task work, 352–353
organizing workflow for fast feedback, 46–47
parallel work and swarming, 350–352
release flow management, 110–111
Scrum used in organizing work flow, 3
sprint flow management, 111–112, 349–350
who does the work, 354
Focus
of development team, 207–208
of sprint retrospective, 378–379
Forecasts
defined, 408
terminology choices for sprint planning outcomes, 17–18
velocity, 135
vs. commitments, 346
Formality
minimizing unnecessary, 57–58
plan-driven development compared with agile development, 60
unnecessary formality defined, 420
Framework, for Scrum
activities and artifacts, 16–18
closing review, 28
core values, principles, and practices in, xxix
daily scrum, 23–25
defined, 408, 416
overview of, 13
practices, 14
product backlog, 18–20
roles, 14–16
sprint execution, 23
sprint planning, 21–23
sprint results, 25–26
sprint retrospective, 27–28
sprint review, 26–27
sprints, 20–21, 61
Frustration, technical debt resulting in, 144
Functional managers. *See* Managers
Funding, incremental/provisional approach to, 304–305

G

Gantt chart
sprint execution and, 349
of up-front plan, 250
Go/no-go decision making
economic filter for, 275–276
funding decisions, 299
Goals. *See also* Sprint goal
managers providing team goals, 228
no end state for, 395
Grenning, James, 129
Grooming
defined, 408
insight backlog, 390
overview of, 104
product backlog, 19, 315
responsibilities of development team, 197
responsibility of product owner, 169
in Scrum framework, 17
ScrumMaster working with product owner on, 190
what it is, 104–105
when does it occur, 106–108
who does it, 105–106
Groups
compared with teams, 209–210
defined, 408
managers role in aligning internal, 234

H

Happened-upon technical debt, 155, 158–159, 408
Harvesters (Goldberg and Rubin), 217
Hidden agendas, transparency and, 189
Hierarchical product backlogs, for large products, 114–115